Sorting Algorithms

Frequency Sort

Core Idea

Count how many times each element appears, then **sort elements based on their frequency** rather than their value. It is not a standalone algorithm — it's a **problem pattern** that combines a HashMap for counting and either a Heap, Bucket Sort, or custom comparator for sorting by frequency.

> Frequency Sort: Doesn't care about the value at all. Cares about **how many times it shows up**. Most frequent to least frequent.

The Two Common Variants

- **Sort by decreasing frequency** — most frequent elements come first
- **Sort by increasing frequency** — least frequent elements come first

---

Steps

1. **Count** the frequency of each element using a HashMap
2. **Sort** elements based on their frequency using one of three approaches:
    - Custom comparator with built-in sort
    - Max Heap (priority queue)
    - Bucket Sort
3. **Build** the result from the sorted frequencies

Visualizing It

```
[4, 2, 2, 8, 3, 3, 3, 1] — sort by decreasing frequency
```

Step 1 — Count frequencies:

```
1 → 1 time
2 → 2 times
3 → 3 times
4 → 1 time
8 → 1 time
```

Step 2 — Sort by frequency (descending):

```
3 (×3) → first
2 (×2) → second
1 (×1) → third
4 (×1) → third
8 (×1) → third
```

Step 3 — Build result:

```
[3, 3, 3, 2, 2, 1, 4, 8] ✅
```

Three Approaches to Sort by Frequency

Approach 1 — HashMap + Custom Sort

Count with a HashMap, then sort using a custom comparator that compares frequencies instead of values.

Time: O(n log n) | Space: O(n)

Approach 2 — HashMap + Max Heap

Count with a HashMap, push all entries into a Max Heap ordered by frequency. Extract one by one to build the result.

Time: O(n log k) | Space: O(n)

> `k` = number of unique elements. Faster than full sort when k is small.

Approach 3 — HashMap + Bucket Sort

Count with a HashMap, create buckets indexed by frequency, place elements in their frequency bucket, then read buckets in reverse order.

Time: O(n) | Space: O(n)

> The most optimal approach — same idea as Bucket Sort.

Visualizing Bucket Sort Approach

```
[4, 2, 2, 3, 3, 3, 1]
```

Buckets indexed by frequency:

```
bucket[1]: [4, 1]   ← appeared once
bucket[2]: [2]      ← appeared twice
bucket[3]: [3]      ← appeared three times
```

Read buckets in reverse → `[3, 3, 3, 2, 2, 4, 1]` ✅

Key Characteristics

- **Not a standalone algorithm** — it's a pattern combining HashMap + sorting technique
- **Stable or not** — depends on which approach you use
- **Flexible** — same pattern works on arrays, strings, and characters

---

Time & Space Complexity

Approach	Time	Space
HashMap + Sort	O(n log n)	O(n)
HashMap + Max Heap	O(n log k)	O(n)
HashMap + Bucket Sort	O(n)	O(n)

Common Interview Problems Using This Pattern

- **Sort Characters by Frequency** — sort a string in decreasing order based on character frequency LeetCode
- **Sort Array by Increasing Frequency** — sort an array in increasing order of frequency, breaking ties by decreasing value LeetCode
- **Top K Frequent Elements** — find the K most frequent elements in an array
- **Sort Elements by Frequency** — sort array so higher frequency elements come first

Why It's Asked in Interviews

It tests whether you can **combine multiple data structures** — HashMap for counting and Heap or Bucket Sort for ordering. Interviewers also use it to see if you can **identify the optimal approach** among the three, and whether you know when O(n) Bucket Sort beats O(n log n) custom sort.

Frequency Sort vs Counting Sort

The One Line Difference

- **Counting Sort** — sorts elements by their **value**
- **Frequency Sort** — sorts elements by their **frequency (count)**

Side by Side on the Same Input

Input: `[4, 2, 2, 3, 3, 3, 1]`

Counting Sort output — sorted by value:

```
[1, 2, 2, 3, 3, 3, 4]
```

Frequency Sort output — sorted by frequency:

```
[3, 3, 3, 2, 2, 1, 4]
```

> 3 appears most (×3), 2 appears next (×2), 1 and 4 appear once (×1)

How Each Uses the Count Array Differently

Counting Sort — count array tells you **where to place each value** in the output

```
count: [0, 1, 2, 3, 1, 0, 0]
         ↑
    use count to place values in sorted order
```

Frequency Sort — count array tells you **how important each element is**

```
count: {1:1, 2:2, 3:3, 4:1}
              ↑
    use count as the sorting key, not the placement guide
```

Key Differences

	Counting Sort	Frequency Sort
Sorts by	Value	Frequency of value
What the count does	Determines position	Becomes the sort key
Is it a full algorithm	✅ Yes	❌ No, it's a pattern
Data structure used	Count array only	HashMap + Heap/Bucket/Sort
Works on	Integers in known range	Any data type
Output goal	Ascending order of value	Descending order of frequency

	Counting Sort	Frequency Sort
Time complexity	O(n + k)	O(n) to O(n log n)

The Simple Mental Model

> **Counting Sort** answers — *"where does each value belong?"*

> **Frequency Sort** answers — *"which value showed up the most?"*

They both **count** things — but what they **do with that count** is completely different.

Counting Sort — *"arrange values in order"*

`[4, 2, 2, 3, 1] → [1, 2, 2, 3, 4]`

Cares about the **value itself**. Smallest to largest.

Frequency Sort — *"arrange by how many times each appears"*

`[4, 2, 2, 3, 3, 3, 1] → [3, 3, 3, 2, 2, 1, 4]`

Doesn't care about the value at all. Cares about **how many times it shows up**. Most frequent to least frequent.

Think of it like a popularity contest. Frequency Sort asks **"who showed up the most?"** and puts them first. The actual value (1, 2, 3, 4...) is irrelevant — only the **count matters**.